MSAS – Assignment #2: Modeling

Alejandro Rivera Míguez, 276270

Exercise 1

To perform a preliminary analysis and evaluate a proposed satellite configuration, it is requested to model an active thermal control system and its associated mechanisms to assess the system's capability to maintain the temperatures within a certain range $[T_{min}; T_{max}]$. The satellite body is a rectangular cuboid (1.5 [m] height, 0.5 [m] side), with fixed solar panels (0.5 [m] side, 0.95 [m] length) located on the +Y-axis. The satellite flies with a fixed inertial attitude, with the satellite's Y-axis always directed towards the Sun. The satellite has two extendable radiators, which are located on the ±X-axis faces. The radiators are connected to the satellite body with electrically controlled hinges that can rotate along the Z-axis direction. The satellite with its radiators in open and closed configurations is represented in Figure 1.

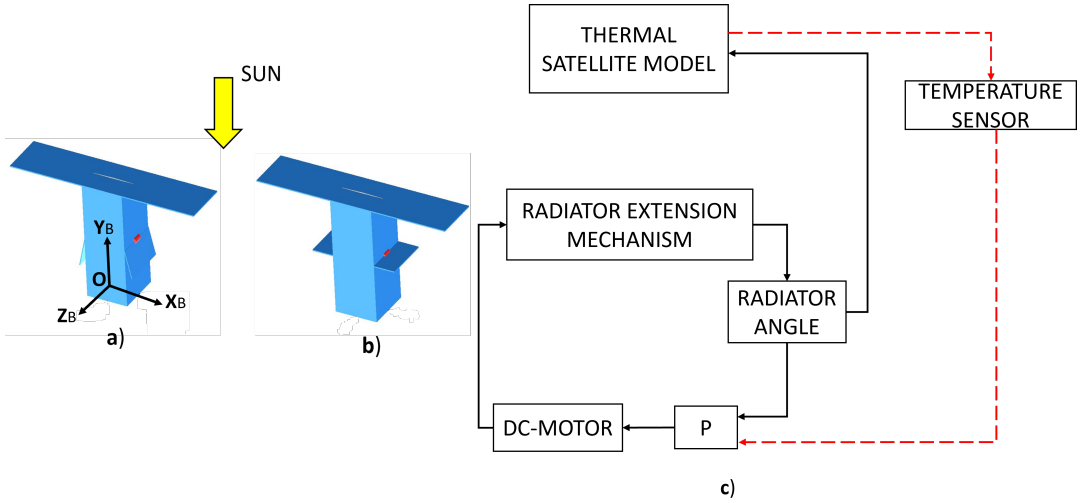


Figure 1: Satellite configuration: **a**) Closed radiator; **b**) Open radiator; **c**) Thermal control logic overview

The extension mechanism is based on a DC motor which gives the needed torque to move the radiator to the desired position. In Figure 2 the physical model of the mechanism is shown. The angle θ represents the rotation of the radiator with respect to the Z-axis and is measured counter-clockwise. For a rotation angle $\theta = 0$ deg, the radiators are in fully open configuration, for $\theta = -0.4\pi$ the radiators are in stowed (closed) configuration. The main parameters of the electro-mechanical model are reported in Table 1.

Parameter	Symbol	Value	Units
Resistance	R	0.1	Ω
Inductance	L	0.001	H
Motor constant	k_m	0.3	Nm/A
Radiator mass	m_r	0.2	kg
Radiator length	L_r	0.5	m

Table 1: Characteristics of the thermal control mechanism.

The radiator has a lower emissivity side, facing deep space in the closed configuration, and a higher emissivity side which is hidden when the radiator is in the stowed position. It is assumed

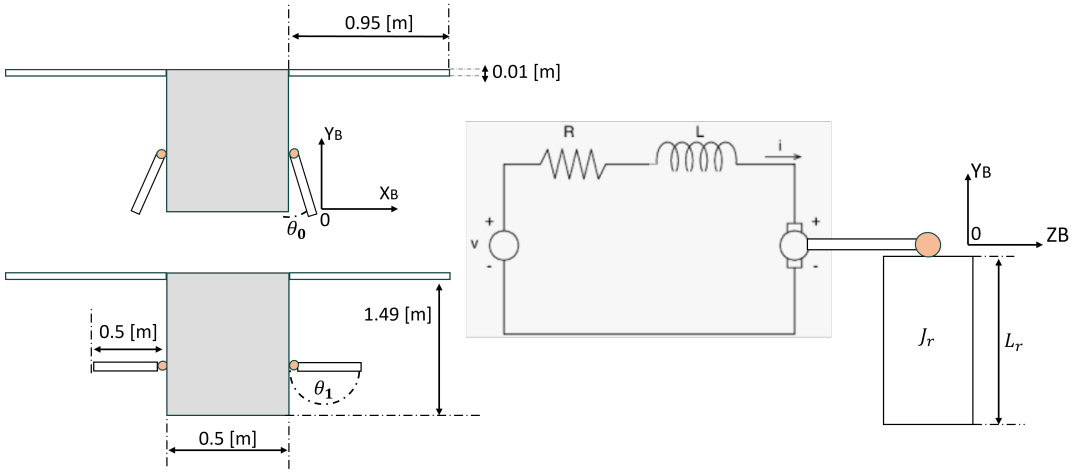


Figure 2: Schematics of the radiator extension mechanism

that radiative heat exchange among the satellite surfaces (re-radiation) is negligible and that no other thermal radiation sources (e.g., planets) are present. As a result, the thermal control of the satellite is regulated mainly by the radiator angle θ, as shown in Figure 1 c). By opening and closing the radiator, the effective area of the high- and low-emissivity sides exposed to deep space increases and decreases respectively. This results in a linear variation of the emissivity ϵ as a function of θ that can be expressed as in Eq. (1):

$$\epsilon(\theta) = \epsilon_{min} + \left[\frac{\epsilon_{max} - \epsilon_{min}}{0.4 \cdot \pi}\right](\theta(t) + 0.4 \cdot \pi) \tag{1}$$

When the radiator is in a closed configuration with $\theta(t) = -0.4\pi$ [rad] the emissivity is at its minimum $\epsilon_{min} = 0.01$. In fully open configuration instead, when $\theta(t) = 0$ [rad], the radiator emissivity reaches its maximum value $\epsilon_{max} = 0.98$. Using a lumped nodes thermal modeling, the satellite can be divided into 5 components (the two solar panels, the main body, and the two radiators). The radiators and solar panels are connected to the main body node by a conductive path, while all nodes are thermally radiating towards deep space (see Figure 3).

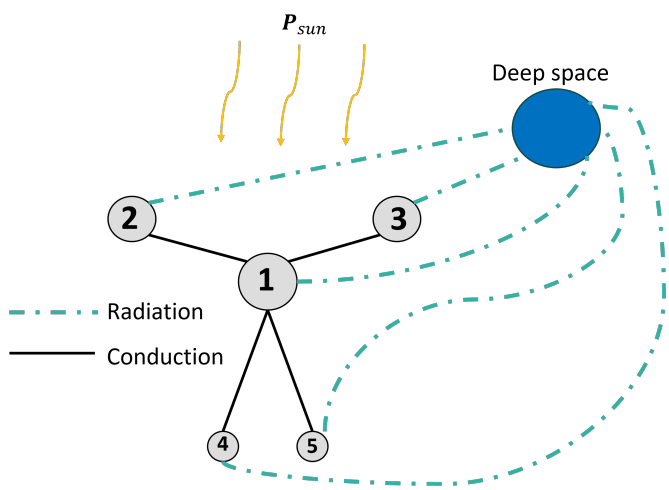


Figure 3: The thermal lumped nodes model

The solar radiation from the Sun is acting only on the two solar panel nodes and on the top face of the main-body node (the radiators are always in shadow). The temperature of deep space is constant and equal to T_{ds}= 3 [K]. The characteristics of the thermal network and its components are given in Table 2.

Parameter	Symbol	Value	Unit
Sun Power	P_{Sun}	1350	W/m^2
Heat capacity	C_1	1.5×10^5	J/K
Heat capacity	C_2, C_3	1187.5	J/K
Heat capacity	C_4, C_5	30	J/K
Thermal conductance	G_{12}, G_{13}, G_{14}, G_{15}	10	W/K
Absorptivity	α_1	0.6	-
Absorptivity	α_2, α_3	0.78	-
Emissivity	ϵ_1	0.45	-
Emissivity	ϵ_2, ϵ_3	0.75	-

Table 2: Thermal parameters of the system.

Part 1: causal modeling (9 points)

Considering the following main constitutive equations of the system:

$$C_i \frac{dT_i}{dt} = \sum_i^n (Q_{in}^i - Q_{out}^i) \tag{2}$$

$$V_{in} = k_p \cdot (T_1 - T_1^{ref}) \tag{3}$$

$$J_r \ddot{\theta} = \tau_{in} \tag{4}$$

where Eq. (2) is the conservation of energy at a thermal node; Eq. (3) is a proportional control law in which V_{in} is the input voltage applied on the DC-motor, and T_1^{ref} is the reference temperature of the main-body; and Eq. (4) is the simplified rotational radiator dynamics where the input torque is linked to current flowing into the motor through the relation: $\tau_{in} = k_m \cdot i$. Assuming that: the upper limit $\hat{T}_{max}$ and lower limit temperature $\hat{T}_{min}$ allowed for **node 1** are 300 [K] and 290 [K] respectively, the motor control is capable of maintaining the angle θ within the physical limits, all the thermal nodes start with a temperature of 298.15 [K], and that the radiator is closed at the beginning of the simulation ($\theta(0) = -0.4\pi$):

1. Formulate the full system of non-linear ODEs making explicit the state variables of the whole simulation.

2. Considering a simulation time of at least 50 [h], define the value for the proportional gain k_p such that: the temperature is kept around the target $T_1^{ref} = 294.15$ [K], and within $t = 10$ [h] the maximum temperature oscillations are less than 0.1% of T_1^{ref}. Plot the resulting temperature evolution over time on all thermal nodes.

3. Discuss the results and the ode used in Matlab for the integration.

Part 2: acausal modeling (6 points)

Using the Modelica standard library, reproduce in Dymola the physical model of the thermal-electro-mechanical system described above. You can build your **own model block** in Modelica to implement specific signals such as $\epsilon(\theta)$ or the control strategy. Please note that when you create a new model block (see in Table 3) you can insert input/output variables and your equations (see as an example the **Text View** of the block: *Modelica/Blocks/Math/Add*). Then, simulate it in OpenModelica comparing the results with the ones obtained using the causal model.

(15 points)

Physical element	*Modelica library*
Rigid-body	Modelica/Mechanics/MultiBody/Parts
Revolute joint	Modelica/Mechanics/MultiBody/Parts
Fixed-translational element	Modelica/Mechanics/MultiBody/Parts
DC-motor	Modelica/Electrical/Machines/BasicMachines/DCMachines
Thermal node	Modelica/Thermal/HeatTransfer/Components
Thermal resistances	Modelica/Thermal/HeatTransfer/Components
Logical switch	Modelica/Blocks/Logical/Switch
Hysteresis	Modelica/Blocks/Logical/Switch
Temperature sensor	Modelica/Thermal/HeatTransfer/Sensors
Your model block	File/New/Block

Table 3: Modelica libraries.

Part 1: causal modeling

1. Formulate the full system of non-linear ODEs making explicit the state variables of the whole simulation.

   The thermal and motor control system is modeled using a set of non-linear ordinary differential equations (ODEs). The state variables are defined as: T_1, T_2, T_3, T_4, T_5 (temperatures of the thermal nodes), θ (radiator angle), $\dot{\theta}$ (angular velocity), and i (motor current). The equations governing the system are as follows:

   (a) **Thermal dynamics:** Each node's temperature is influenced by radiative heat transfer (Q_{rad}), conductive heat transfer (Q_{cond}) and nodes 1, 2 and 3 also by solar radiation input (Q_{sun}).

$$Q_{\text{cond,i}} = G_{ij}(T_j - T_i)$$

$$Q_{\text{rad,i}} = \sigma \epsilon_i A_i (T_i^4 - T_{\text{ds}}^4)$$

$$Q_{\text{sun,i}} = P_{\text{sun}} \alpha_i A_i$$

   The general thermal equation for node i is:

$$C_i \frac{dT_i}{dt} = Q_{\text{sun},i} + Q_{\text{cond},i} - Q_{\text{rad},i}$$

   where C_i is the heat capacity of node i.

   - **Node T_1 (main body):** This node receive solar input on its upper surface, exchanges heat with nodes T_2, T_3, T_4 and T_5 and emits radiation outwards:

$$\frac{dT_1}{dt} = \frac{1}{C_1} \left(Q_{\text{sun},1} + \sum_{j=2}^{5} G_{1j}(T_j - T_1) - Q_{\text{rad},1} \right).$$

   - **Nodes T_2 and T_3 (solar panels):** These nodes receive solar input, conduct heat to T_1 and emits radiation outwards:

$$\frac{dT_2}{dt} = \frac{1}{C_2} \left(Q_{\text{sun},2} + G_{12}(T_1 - T_2) - Q_{\text{rad},2} \right).$$

$$\frac{dT_3}{dt} = \frac{1}{C_3} \left(Q_{\text{sun},3} + G_{13}(T_1 - T_3) - Q_{\text{rad},3} \right).$$

- **Nodes T_4 and T_5 (radiators):** These nodes only exchange heat with T_1 and radiate heat:

$$\frac{dT_4}{dt} = \frac{1}{C_4}\left(G_{14}(T_1 - T_4) - Q_{\text{rad},4}\right),$$
$$\frac{dT_5}{dt} = \frac{1}{C_5}\left(G_{15}(T_1 - T_5) - Q_{\text{rad},5}\right).$$

(b) **Motor dynamics:** The radiator angle θ is controlled by a motor. The angular acceleration is given by:

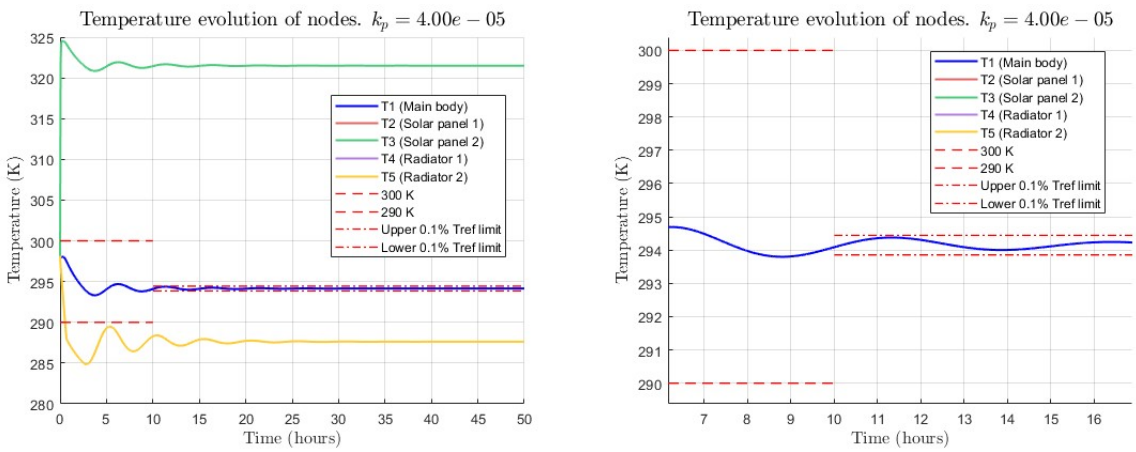
$$\ddot{\theta} = \frac{k_m i}{J_r},$$

where k_m is the motor constant, i is the current and J_r is the moment of inertia. The current i evolves according to:

$$\frac{di}{dt} = \frac{1}{L}\left(k_p(T_1 - T_1^{\text{ref}}) - Ri - k_m\dot{\theta}\right),$$

where k_p is the proportional gain, R the motor resistance and L the inductance.

2. Considering a simulation time of at least 50 [h], define the value for the proportional gain k_p such that: the temperature is kept around the target $T_1^{ref} = 294.15$ [K], and within $t = 10$ [h] the maximum temperature oscillations are less than 0.1% of T_1^{ref}. Plot the resulting temperature evolution over time on all thermal nodes.



(a) General. **(b)** Enlargement at t = 10 h.

Figure 4: Temperature evolution over time for all thermal nodes.

From the simulation results:

- **Optimal Gain:** $k_p = 4 \times 10^{-5}$.

Temperature Evolution Plot: Figure 4 shows that the main body temperature T_1 stabilizes around 294.15 K within the specified tolerance. Throughout the simulation, the temperature consistently remains within the allowable range of 290 K to 300 K. Additionally, the maximum temperature oscillations are kept below 0.1% of T_1^{ref} (see Figure 4b for a detailed view).

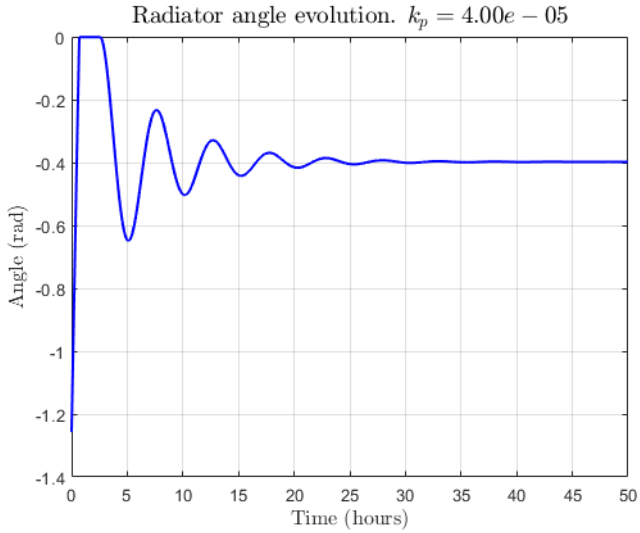


Figure 5: Radiator angle evolution over time.

Radiator Angle Evolution: Figure 5 shows the evolution of the radiator angle θ in response to the control signal. The angle remains within the range $[-0.4\pi, 0]$ throughout the simulation. The motor voltage is applied to adjust the angle, but it is cut off when the radiator reaches the fully open position ($\theta = 0$) and remains off until the temperature of node 1 reaches 294.15 K, allowing the system to stabilize effectively at the desired temperature.

3. Discuss the results and the ode used in Matlab for the integration.

   The simulation results confirm that the selected proportional gain ($k_p = 4 \times 10^{-5}$) ensures the stabilization of T_1 around the target temperature $T_1^{\text{ref}} = 294.15$ K within 10 hours, with maximum oscillations below 0.1%. The radiator angle θ operates effectively within the expected range, maintaining thermal control without exceeding physical constraints. The results validate the model's capability to regulate the thermal environment under specified conditions.

   The selected solver for integrating the system's non-linear ODEs was MATLAB's `ode23t`, which is well-suited for stiff problems due to its implementation of the trapezoidal rule with a backward differentiation formula (BDF). Stiffness arises in this system from the coupling between the thermal dynamics and the motor control, which involve timescales of different orders of magnitude. Other solvers, such as `ode45`, a Runge-Kutta method for non-stiff systems, or `ode15s`, a BDF solver for stiff systems, could have been used. However, `ode45` proved inefficient due to the stiffness of the equations, requiring excessively small time steps to maintain stability. The choice of `ode23t` strikes a balance between computational efficiency and accuracy, making it ideal for this specific problem.

Part 2: Acausal Modeling (6 points)

Using the Modelica Standard Library, the physical model of the thermal-electro-mechanical system described above was reproduced in **Dymola**. The simulation results were then compared with those obtained from the causal model in MATLAB and OpenModelica.

This exercise is divided into two parts:

1. The creation of the Dymola model with brief descriptions of each block.
2. A comparison of the simulation results from MATLAB and OpenModelica.

Description of the Simscape Model

The Simscape model consists of two main components: the **Thermal model** and the **Engine dynamics model**.

- **Thermal Model:**

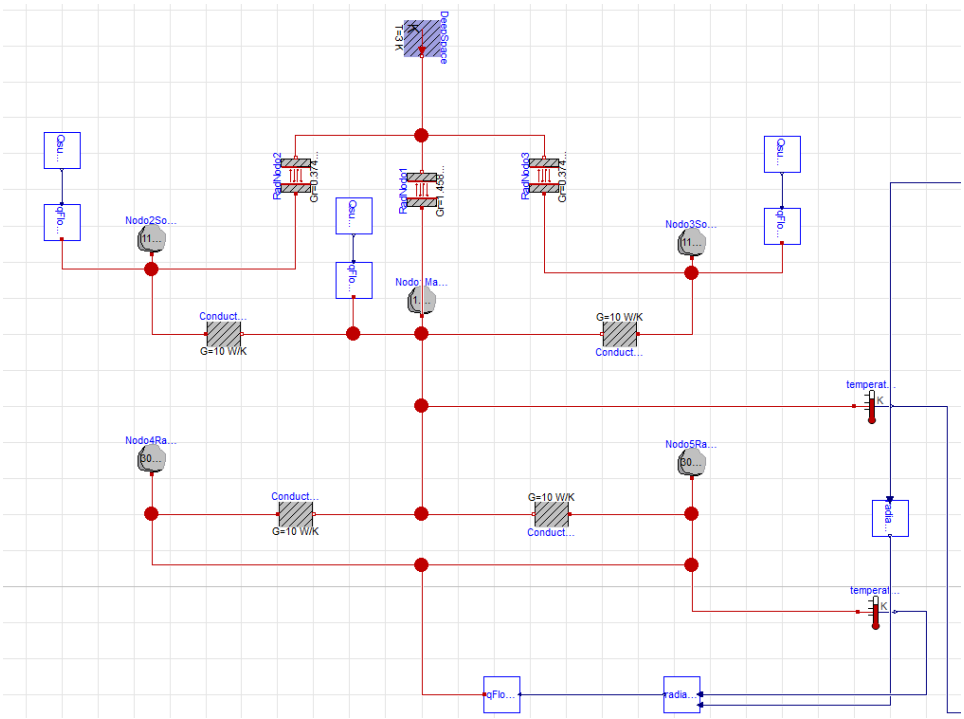


Figure 6: Thermal model implemented in Dymola.

The thermal model represents the satellite's temperature distribution using lumped thermal nodes.

Model Description

The system consists of five thermal nodes, each modeled as a *Heat Capacitor*. Key elements include:

- **Thermal Capacitance** (C): Represents heat storage for each node.
- **Thermal Conductance** (G): Represents conductive heat transfer between nodes.
- **Radiative Exchange**: Each node radiates heat to deep space at $T_{ds} = 3\,\mathrm{K}$.

- **Solar Absorption:** Solar radiation affects Nodes 2, 3, and the top of Node 1. This effect is modeled using a custom block (*Qsun*) that defines the equation:

$$Q_{\text{sun}} = P_{\text{sun}} \alpha A$$

- **Radiator Radiation:** Radiator radiation is modeled using two custom blocks:
  1. **Radiator Emissivity Block** (*RadiatorEmissivity*):
     - **Input:** Radiator angle θ (measured using an angle sensor).
     - **Output:** Emissivity $\epsilon(\theta)$, which varies linearly with θ.
  2. **Radiator Radiation Block** (*RadiatorRadiation*):
     - **Inputs:** Node temperature and radiator emissivity.
     - **Output:** Heat flux due to radiation.

Operation Summary

The thermal model simulates the satellite's heat distribution using lumped thermal nodes. Heat is stored in capacitors, transferred between nodes via thermal conductance, and radiated to deep space at $T_{ds} = 3\,\text{K}$. Solar absorption increases the temperature of specific nodes, while the radiator dissipates heat based on its angle-dependent emissivity. These interactions collectively determine the thermal dynamics of the system.

- **Engine dynamics model:**

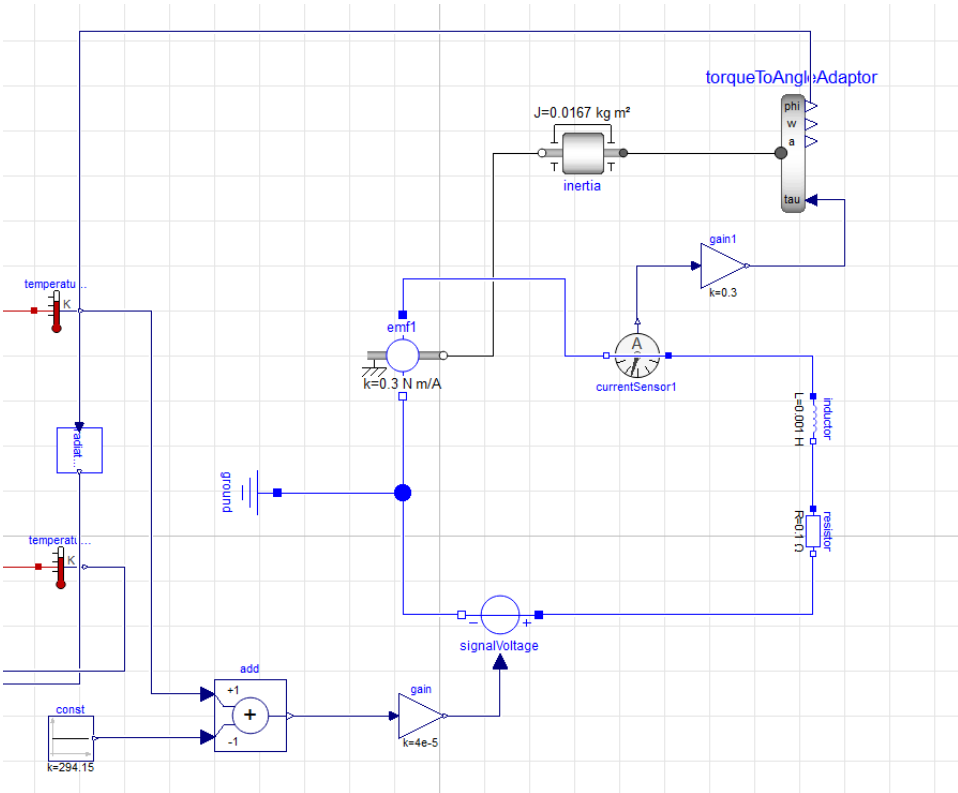


Figure 7: Engine dynamics model implemented in Dymola.

Model Description

The key elements of this model include:

- **Electromotive Force (EMF)**: Converts electrical current into torque based on the relationship:

$$T = k \cdot I$$

where $k = 0.3\,\mathrm{N{\cdot}m/A}$.

- **Inertia Block**: Represents the rotational inertia of the motor and the load. Moment of inertia $J = 0.0167\,\mathrm{kg{\cdot}m^2}$.
- **Torque to Angle Adaptor**: Converts the torque input into angular position (ϕ) and velocity (ω).
- **Current Sensor**: Measures the current flowing through the motor circuit.
- **Electrical Components**:
  * **Inductor**: $L = 0.001\,\mathrm{H}$
  * **Resistor**: $R = 1\,\Omega$

  These components model the electrical characteristics of the motor winding.
- **Gain Blocks**:
  * *gain*: $k_p = 4 \times 10^{-5}$ (scales the input voltage signal, same value as in MATLAB)
  * *gain1*: $k = 0.3$ (scales the torque output)
- **Signal Voltage Source**: Provides the input voltage to the motor circuit.
- **Radiator Emmisivity** (*radiatorEmissivity*): It is a custom block that receives the θ angle as input and returns the emissivity (ϵ) of the radiators.
  * **Input:** Radiator angle θ (measured using an angle sensor).
  * **Output:** Emissivity $\epsilon(\theta)$, which varies linearly with θ.

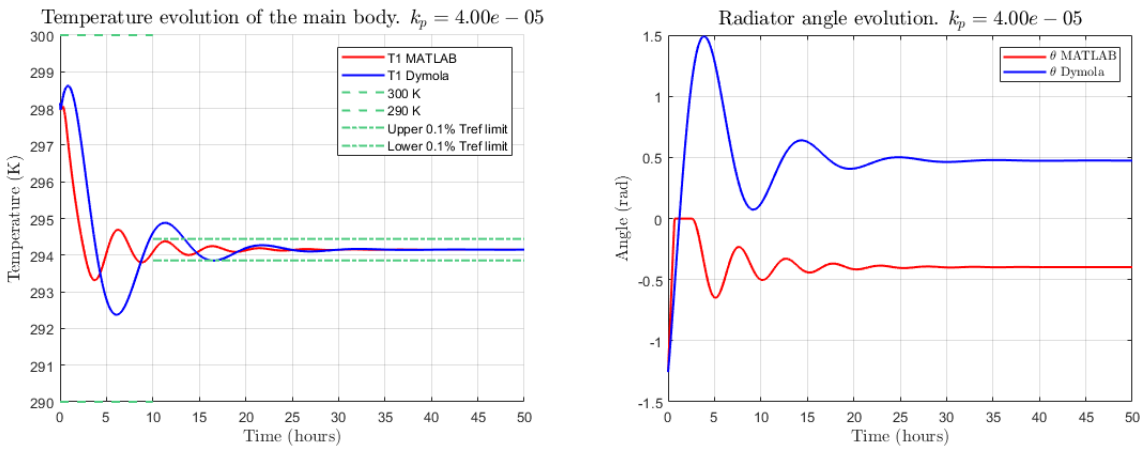
Operation Summary

The voltage applied to the motor circuit generates a current, which is converted into torque by the EMF block. This torque acts on the inertia block, causing rotational motion. The torque-to-angle adaptor converts this motion into angular position and velocity, closing the loop of the engine dynamics simulation.

Note: The *RadiatorEmissivity* block serves as the coupling element between the mechanical and thermal subsystems. By receiving the radiator angle θ from the mechanical model and outputting the angle-dependent emissivity ϵ to the thermal model, it ensures a dynamic interaction between these two domains.

Comparison of the Simulation Results

The simulation results obtained from MATLAB and Dymola for the temperature evolution of the main body (T_1) and the radiator angle (θ) are presented below. Despite multiple attempts, I was unable to control the radiator angle (θ) in Dymola within the desired range $[-0.4\pi, 0]$. Using the same proportional gain ($k_p = 4.00 \times 10^{-5}$) in both simulations resulted in significant discrepancies in θ, affecting the thermal behavior of the system.



(a) Temperature evolution of the main body (T_1). **(b)** Radiator angle evolution (θ).

Figure 8: Simulation results comparing MATLAB and Dymola.

- **Temperature Evolution of the Main Body (T_1):** Figure 8a shows that in MATLAB, the temperature (T_1) stabilizes predictably. In Dymola, deviations occur because θ could not be controlled within the desired range. As a result, T_1 exceeds the upper limit after 10 hours. Proper control of θ in Dymola would likely produce a temperature profile similar to MATLAB's.
- **Radiator Angle Evolution (θ):** Figure 8b shows that in MATLAB, θ stays within the desired range $[-0.4\pi, 0]$, ensuring consistent thermal dissipation. In Dymola, θ overshoots significantly and fails to stay within limits, despite several adjustments. This poor control of θ negatively affects the system's thermal performance.

Conclusion: The inability to control θ in Dymola directly impacts T_1. Despite using the same proportional gain and making several adjustments, θ exceeded limits in Dymola, causing T_1 to surpass the upper reference limit after 10 hours. In MATLAB, where θ was successfully controlled, the thermal behavior remained stable.

Exercise 2

A simplified conceptual map of the Attitude Control System (ACS) of a satellite is depicted in Figure 9. For the accuracy of some measurements that the spacecraft has to obtain, the probe has to fly in an orbit very close to the Earth's surface. The spacecraft is therefore subjected to a high atmospheric drag. The ACS has to compensate for the deceleration caused by this drag continuously. In general, ACSs are made of 3 sensors in orthogonal directions and 3 thrusters to detect and compensate for linear and angular accelerations on the probe in each direction, but for the sake of this project, only compensation in the tangential direction will be considered.

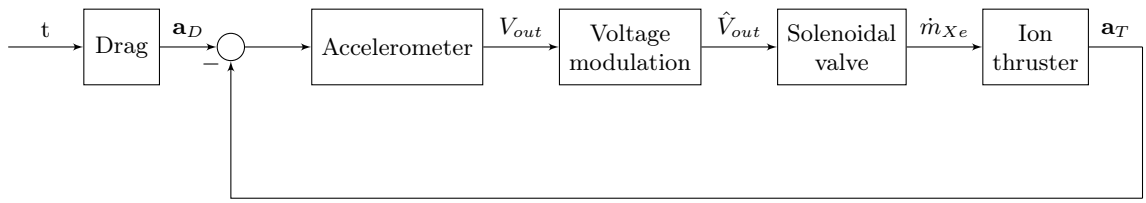


Figure 9: Block diagram of the ACS.

The ACS is modeled with several modules: an **accelerometer** measuring the accelerations acting on the spacecraft, a **voltage modulation block** controlling the output voltage, the **control valve** modifying the aperture of the thruster valve, and the **ion thruster**.

The accelerometer, kept aligned with the direction of velocity, presents inside a seismic mass that is subject to an external acceleration $\mathbf{a_D}$ and $\mathbf{a_T}$ given only by the drag D and the thrust T, respectively. The mass is in between the stators of a condenser and, moving for the acceleration given by these two forces changes the output voltage V_{out} of the circuit of the accelerometer. The dynamics of the seismic mass itself can be modeled as a mass-spring-damper system as follows

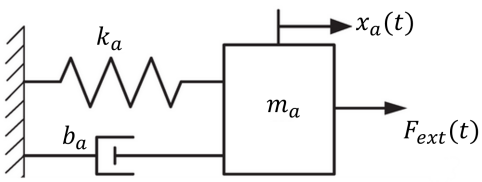


Figure 10: Model of the accelerometer.

$$\dot{x}_a = v_a \tag{5}$$

$$F_{ext}(t) = \frac{T - D}{M_{SC}} m_a = m_a \dot{v}_a + b_a v_a + k_a x_a \tag{6}$$

where M_{SC} is the mass of the spacecraft, m_a is the seismic mass value, and b_a and k_a are respectively the damper and the spring terms present in the accelerometer. For the causal modeling, consider the V_{out} as directly proportional to the velocity of the seismic mass, as $V_{out} = K_{acc} v_a$ [1].

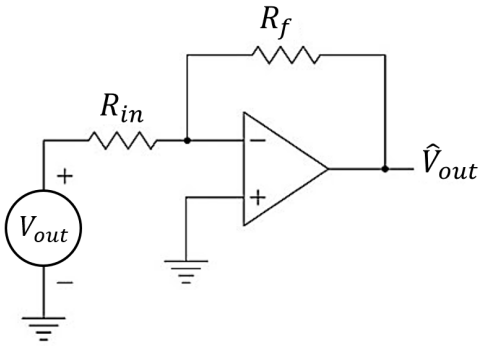


Figure 11: Inverting operational amplifier.

The output of the accelerometer V_{out} needs to be modulated into $\hat{V}_{out}$ to adjust the control for the solenoidal valve. This modulation is performed through an operational amplifier, displayed in Figure 11. The operational amplifier in inverting configuration modifies V_{out} into $\hat{V}_{out}$ accordingly to:

$$\hat{V}_{out} = -\frac{R_f}{R_{in}} V_{out} \tag{7}$$

where R_f and R_{in} are the two resistances modifying the input voltage.

The voltage $\hat{V}_{out}$ is itself the input of the solenoidal valve, shown in Figure 12, and creates a current I commanding the flow control valve. The current passes through a circuit modeled with only a solenoid with a variable inductance $L(x_v)$. As the current variably flows, a magnetic field is generated. A resulting force f_v acts on the spool, which as a result moves.

The armature-spool arrangement is then modeled again as a lumped parameter spring-mass-damper system

$$\dot{x}_v = v_v \tag{8}$$

$$m_v \dot{v}_v = -k_v x_v - b_v v_v + \frac{1}{2} I^2 \frac{dL}{dx_v} \tag{9}$$

$$\dot{I} = \frac{1}{L(x_v)} \hat{V}_{out} \tag{10}$$

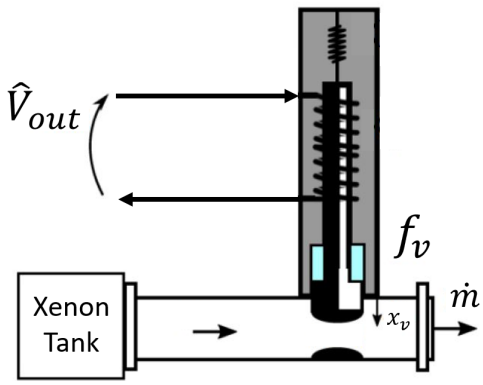


Figure 12: Model of the solenoidal valve.

where x_v and v_v represent the position and velocity of the spool, and b_v and k_v are respectively the damper and the spring terms, and m_v is the mass of the spool. The dynamics of the current I is generated by $\hat{V}_{out}$ through the variable inductance $L(x_v) = \frac{1}{\alpha+\beta x_v}$, depending on the position of the valve itself.

As the spool moves, the area $A_v(x_v)$ of the duct through which a Xenon flow is passing changes. The resulting mass flow rate $\dot{m}_{Xe}$ of Xenon is ejected into the ion thruster, shown in Figure 13. In particular, the area of the valve can be modeled as

$$A_v(x_v) = A_0 + \ell(x_{v,\max} - x_v) \tag{11}$$

In particular, the orifice area $A_v(x_v)$ has been modeled as linearly varying with x_v. The tool controlling the area is a flap of width ℓ and maximum extension $x_{v,\max} = \ell$, for simplicity.

The value A_0 is the minimum area of the orifice, such that:

$$0 \leq x_v \leq x_{v,\max}$$

$$\underbrace{A_0 = A_v(x_{v,\max})}_{\text{minimum area}} \leq A_v \leq \underbrace{A_v(0) = A_0 + \ell^2}_{\text{maximum area}} \tag{12}$$

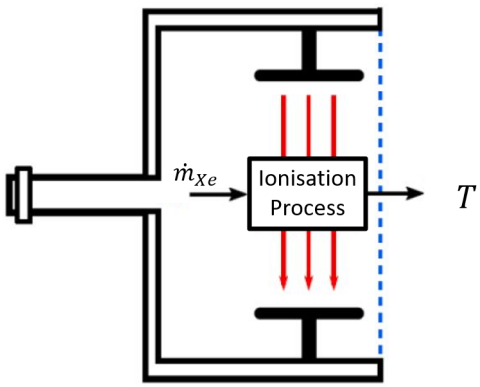


Figure 13: Model of the ion thruster.

The ion thruster comprises an ionization chamber that utilizes a strong magnetic field, ionizing the Xenon flux. The generated ions are moved towards the acceleration grid, which accelerates the ions using a strong electric field, producing a thrust on the spacecraft as the final effect. The flux of Xenon $\dot{m}_{Xe}$ can be expressed as follows

$$\dot{m}_{Xe} = A_v(x_v)\sqrt{k\rho_{Xe}p_T(\frac{2}{k+1})^{\frac{k+1}{k-1}}} \tag{13}$$

where $\rho_{Xe} = \dfrac{p_T}{\bar{R}T_T}$, k and $\bar{R}$ being respectively Xenon specific heat ratio and gas constant. In the tank, the gas is at a total pressure of p_T and temperature of T_T. The Xenon flow enters the

thruster and is here ionized into ions with mass m_i and charge q. After the acceleration imposed by the acceleration grid with a voltage ΔV, the ions exit from the nozzle with a velocity v_{exit}, producing the thrust T, as follows

$$T = \dot{m}_{Xe} v_{ext} = \dot{m}_{Xe} \sqrt{\frac{2q\Delta V}{m_i}} \tag{14}$$

Table 4 reports the values for the parameters to simulate the ACS system, with symbol and unit of measure. The drag $D(t)$ can be modeled as a function of time only, as it follows

$$D(t) = 2.2 - \cos(\omega_s t) + 1.2 \sin(\omega_o t) \cos(\omega_o t) \tag{15}$$

where t enters in seconds, and $D(t)$ is in mN.

Component	**Parameter**	**Symbol**	**Value**	**Unit**
Accelerometer	Spacecraft mass	M_{SC}	300	kg
	Seismic mass	m_a	0.32	kg
	Accelerometer damper	b_a	$[1.5{\cdot}10^3$–$2{\cdot}10^4]$	Ns/m
	Accelerometer spring	k_a	$[5{\cdot}10^{-5}$–$3{\cdot}10^{-3}]$	N/m
	Acc. proportional coefficient	K_{acc}	1	Vs/m
Amplifier	Inverting resistance	R_{in}	[0.1–10]	Ω
	Feedback resistance	R_f	$[1{\cdot}10^4$–$8{\cdot}10^4]$	Ω
Solenoidal Valve	Spool mass	m_v	0.1	kg
	Valve spring	k_v	$1{\cdot}10^3$	N/m
	Valve damper	b_v	$1{\cdot}10^3$	Ns/m
	Solenoid constant	α	$2.1{\cdot}10^{-2}$	1/H
	Solenoid gain	β	-60	1/Hm
	Minimum area	A_0	$4.7{\cdot}10^{-12}$	m^2
	Maximum extension	$x_{v,\max}$	$1{\cdot}10^{-5}$	m
Thruster	Heat ratio	k	1.66	-
	Tank pressure	p_T	$2{\cdot}10^5$	Pa
	Tank temperature	T_T	240	K
	Gas constant	$\bar{R}$	63.32754	J/kg K
	Charge	q	$1.6{\cdot}10^{-19}$	C
	Voltage	ΔV	2000	V
	Ion mass	m_i	$2.188{\cdot}10^{-25}$	kg
Drag	Secular pulsation	ω_s	$1.658226{\cdot}10^{-6}$	rad/s
	Orbital pulsation	ω_o	$1.160758{\cdot}10^{-3}$	rad/s

Table 4: Parameters for the simulation of the ACS.

Part 1: causal modeling (9 points)

Reproduce in Matlab the physical model of the ACS, in particular

1. Formulate the full system of non-linear ODEs making explicit the state variables of the whole simulation. Tune the values of the parameters not explicitly stated in Table 4 to have the compensating thrust T the most similar to the disturbing drag D.

2. Choose the appropriate initial conditions and simulate for three orbital periods T_o, where $T_o = 2\pi/\omega_o$.

3. Discuss the selection of the ODE integration scheme.

Part 2: acausal modeling (6 points)

Reproduce in Simscape the physical model of the ACS. Choose the appropriate initial conditions and parameters, simulate for three orbital periods T_o, and compare the results with those obtained in the prior point.

Note: when modeling the solenoid valve in Simscape, please consider the nominal values shown in Table 5.

Parameter	**Value**	**Unit**
Pull-in forces [F_1 - F_2]	[9000 - 12]	N
Stroke [x_1 - x_2]	[0 - 0.1]	mm
Maximum Stroke	0.1	mm
Rated voltage	0.6	mV
Rated current	0.1	A
Contact stiffness	0.12×10^6	N/m
Contact damping	10^4	Ns/m

Table 5: Solenoid parameters in Simscape.

(15 points)

Part 1: causal modeling (9 points)

Reproduce in Matlab the physical model of the ACS, in particular

1. Formulate the full system of non-linear ODEs making explicit the state variables of the whole simulation. Tune the values of the parameters not explicitly stated in Table 4 to have the compensating thrust T the most similar to the disturbing drag D.

   The system is described using the following state variables:

   - x_a: Displacement of the seismic mass in the accelerometer [m].
   - v_a: Velocity of the seismic mass in the accelerometer [m/s].
   - x_v: Displacement of the valve in the ion thruster [m].
   - v_v: Velocity of the valve in the ion thruster [m/s].
   - I: Current through the solenoid valve [A].

   The coupled non-linear ODEs are formulated as follows:

$$
\begin{aligned}
\dot{x}_a &= v_a, \\
\dot{v}_a &= \frac{T-D}{m_{\text{SC}}} - \frac{b_a}{m_a} v_a - \frac{k_a}{m_a} x_a, \\
\dot{x}_v &= v_v, \\
\dot{v}_v &= \frac{F_v - b_v v_v - k_v x_v}{m_v}, \\
\dot{I} &= \frac{1}{L(x_v)} \hat{V}_{\text{out}}.
\end{aligned}
$$

   Where F_v is:

$$
F_v = -\frac{1}{2} I^2 \frac{\beta}{(\alpha + \beta x_v)^2},
$$

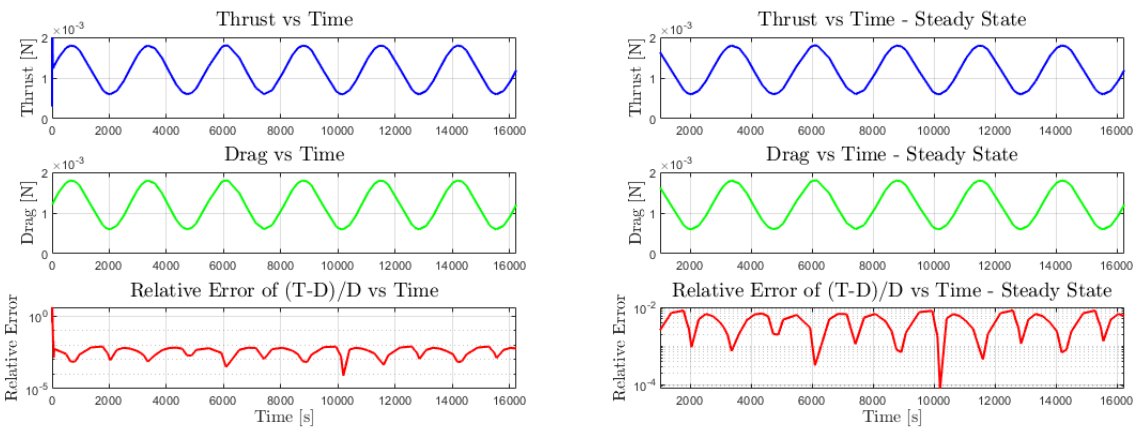
   and $L(x_v)$ y $\hat{V}_{\text{out}}$ are those in the statement.

The following parameters were optimized to achieve the closest match between T and D:

- $b_a = 1500.1719$ Ns/m,
- $k_a = 2.179 \times 10^{-3}$ N/m,
- $R_{\text{in}} = 0.15529\ \Omega$,
- $R_f = 64017.17\ \Omega$.

To obtain these values, the genetic algorithm was employed to explore the solution space efficiently, optimizing each parameter within predefined bounds. The objective was to minimize the relative error between thrust (T) and drag (D) over the entire orbital period. The total computation time required to find the optimal parameters using the genetic algorithm was 9 seconds.

The results are presented in the following figures:



(a) Full simulation. **(b)** Steady-state behavior.

Figure 14: Thrust, Drag, and Relative Error over Time.

The plots depict the thrust and drag forces as oscillatory functions of time. The relative error, shown in the bottom graph, remains below 1% once the transient phase ends and the system reaches steady-state oscillations (see Figure 20b). Initial fluctuations are attributed to transient effects before the system stabilizes.

2. Choose the appropriate initial conditions and simulate for three orbital periods T_o, where $T_o = 2\pi/\omega_o$.

   The selected initial conditions ensure the system begins in a realistic and physically consistent state, with all key variables stable. The chosen values are as follows:

   - The initial position of the seismic mass $x_a(0)$ was set to zero, assuming no prior displacement in the accelerometer.
   - The initial velocity of the seismic mass $v_a(0)$ was also set to zero, considering that the spacecraft starts in a steady-state condition with balanced thrust and drag forces.
   - The spool position $x_v(0)$ in the solenoidal valve was initialized at $x_{v,\max}$, corresponding to the minimum area A_0 of the valve.
   - The initial velocity of the spool $v_v(0)$ was set to zero, assuming no prior motion of the spool.

- The initial current $I(0)$ was set to zero, assuming the system starts at a steady-state equilibrium. At $x_v(0) = x_{v,\max}$, the valve area is at its minimum A_0, requiring no Xenon flow. Since $\hat{V}_{out} = 0$ at steady-state, the solenoid does not provide any current, making $I(0) = 0$ a consistent and realistic initial condition.

3. Discuss the selection of the ODE integration scheme.

   The integration of the system of ODEs was performed using the `ode15s` solver due to its superior stability and accuracy in handling stiff systems. The equations involve fast dynamic transitions (e.g., solenoid valve dynamics) coupled with slower orbital-scale behaviors, which introduce stiffness into the system. Several solvers were tested, including `ode45`, `ode23s`, `ode23t`, and `ode15s`, to determine the most suitable method for this application.

   The `ode45` solver, while efficient for non-stiff systems, proved inadequate for the stiff nature of the problem, resulting in excessive computation times and numerical instabilities. Both `ode23s` and `ode23t`, designed for stiff systems, demonstrated improved stability and handled rapid transitions effectively, but they were less efficient in terms of overall accuracy for the required precision.

   Ultimately, `ode15s` emerged as the optimal solver, offering the best trade-off between accuracy and computational efficiency. Although slightly slower than `ode23s`, `ode15s` provided significantly more accurate results for the system under study, making it the preferred choice for this analysis.

Part 2: acausal modeling (6 points)

Reproduce in Simscape the physical model of the ACS. Choose the appropriate initial conditions and parameters, simulate for three orbital periods T_o, and compare the results with those obtained in the prior point.

This exercise is divided into two parts:

1. The creation of the simscape model, with a brief and concise description of each block.
2. A comparison of the results obtained in MATLAB and Simscape.

Description of the Simscape Model

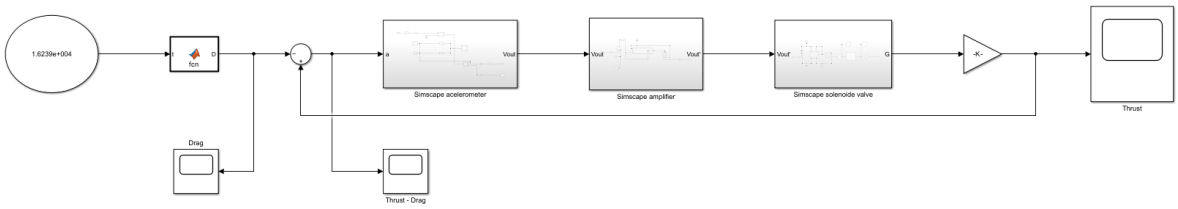


Figure 15: Main block in Simscape.

The Simscape model starts with a MATLAB function that calculates the drag force using the formula provided in the problem statement.

The calculated drag is then used to determine the net force, which is obtained by subtracting the drag D from the thrust force T:

$$F_{net} = T - D.$$

This net force is central to the behavior of the system.

The main block processes the net force input and distributes it to the corresponding subsystems.

- **Accelerometer Subsystem:** The accelerometer subsystem measures the acceleration generated by the net force. It is configured to convert the physical acceleration into an output signal (in this case, voltage) proportional to the acceleration value.

  The accelerometer subsystem is shown in Figure 16.

- **Amplifier Subsystem:** The amplifier subsystem receives the output signal from the accelerometer V_{out} and adjusts it to produce $\hat{V}_{out}$, which is used for controlling the solenoidal valve. The amplifier ensures that the signal is scaled appropriately to meet the system's operational requirements.

  The implementation of the amplifier is presented in Figure 17.

- **Solenoid Valve Subsystem:** The solenoid valve subsystem controls the flow of fluid based on the amplified signal $\hat{V}_{out}$. The valve operates proportionally to the input voltage, regulating the fluid flow and ultimately influencing the system's response.

  The MATLAB function block, which is part of this subsystem, determines the valve's opening area based on the input displacement x_v. This dynamic behavior allows the valve to open or close proportionally, depending on the system's requirements.

  The final gain block calculates the flux of Xenon $\dot{m}_{Xe}$ based on the formula provided in the problem statement.

  The implementation of the solenoid valve subsystem is shown in Figure 18.

- **Ion thruster subsystem:**

  The final subsystem, as shown in Figure 15, calculates the thrust (T) of the system. In this case, there is no need to create a dedicated subsystem since the computation only involves applying the formula provided in the problem statement:

$$T = \dot{m}_{Xe} v_{ext}$$

  The input signal to this block is the flux of Xenon $\dot{m}_{Xe}$, which has been calculated in the solenoid valve subsystem. By applying the provided formula, the thrust is determined directly using a gain.

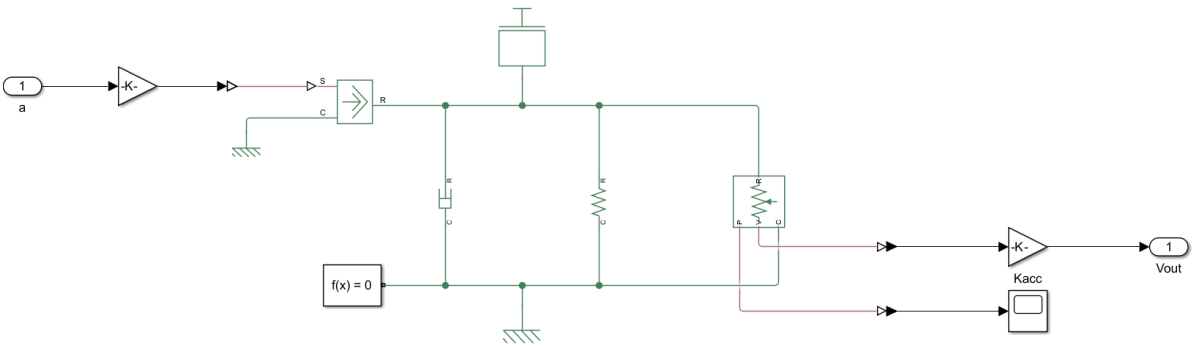


Figure 16: Accelerometer subsystem configuration.

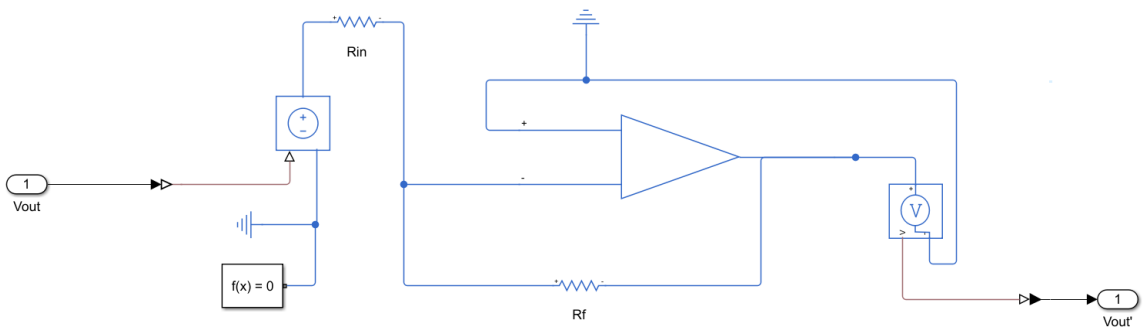


Figure 17: Amplifier subsystem implementation.

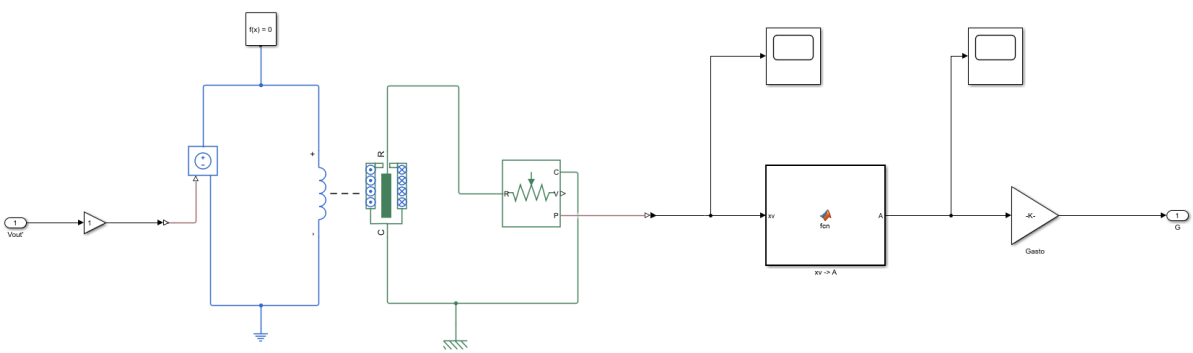


Figure 18: Solenoid valve subsystem configuration.

Comparison of Results: MATLAB vs Simscape

The comparison will initially be conducted using identical parameters for MATLAB and Simscape. Subsequently, a genetic algorithm will be applied in Simscape, and the results will be analyzed and evaluated.

- **Comparison with identical parameters**
  Identical parameter values were employed in both simulations to ensure a fair comparison between MATLAB and Simscape. This approach isolates any differences to the modeling methodologies and solver implementations. In both cases, the chosen ODE solver was `ode15s`.

  - $b_a = 1500.1719$ Ns/m,
  - $k_a = 2.179 \times 10^{-3}$ N/m,
  - $R_{\text{in}} = 0.15529\ \Omega$,
  - $R_f = 64017.17\ \Omega$.

  The initial conditions are slightly different:

  Table 6: Initial Conditions for MATLAB and Simscape Simulations

Variable	MATLAB	Simscape
$x_a(0)$	0	0
$v_a(0)$	0	0
$x_v(0)$	$x_{v,\max}$	0
$v_v(0)$	0	0
$I(0)$	0	0

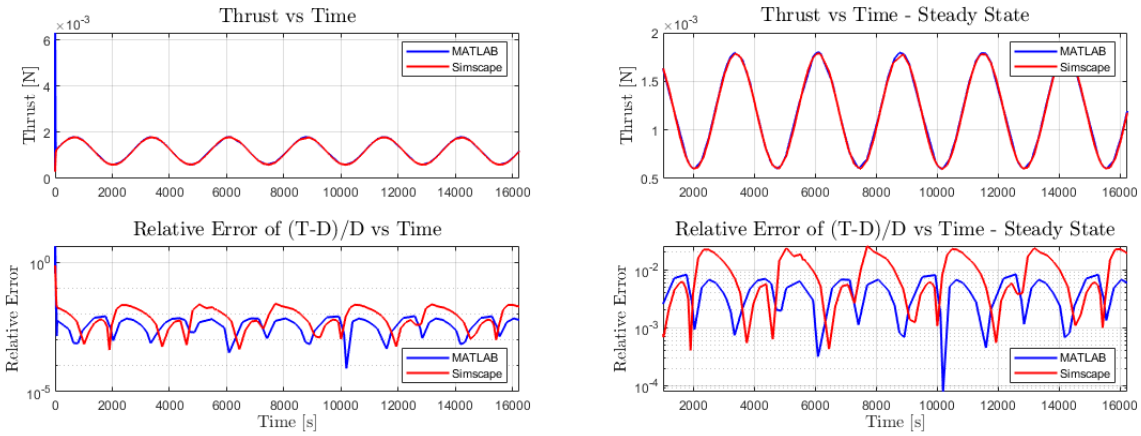
  The initial condition for x_v differs between MATLAB and Simscape due to a difference in how the valve position is defined in each platform. In Simscape, $x_v = 0$ represents the valve fully closed, while in MATLAB, $x_v = x_{v,\max}$ corresponds to the minimum valve area A_0. To ensure consistency, a MATLAB Function block in Simscape is placed after the solenoidal to add this small area A_0 (see Figure 18).

  The comparison focuses on the thrust generated by the ion thruster and the relative error $(T - D)/D$ over time, as illustrated in Figure 20.

  Note that since the drag equation is identical in both implementations, it has been decided not to consider it for comparing the results.

  In the full simulation (Figure 20a), transient behavior leads to higher discrepancies between MATLAB and Simscape. Once steady-state is reached (Figure 20b), the thrust values converge with minimal differences. Relative errors during steady-state are predominantly low (1% to 0.1%), with occasional spikes in Simscape ($> 1\%$) attributed to numerical precision.

  These results validate Simscape's ability to accurately replicate MATLAB's ACS system dynamics.



(a) Full simulation. **(b)** Steady-state behavior.

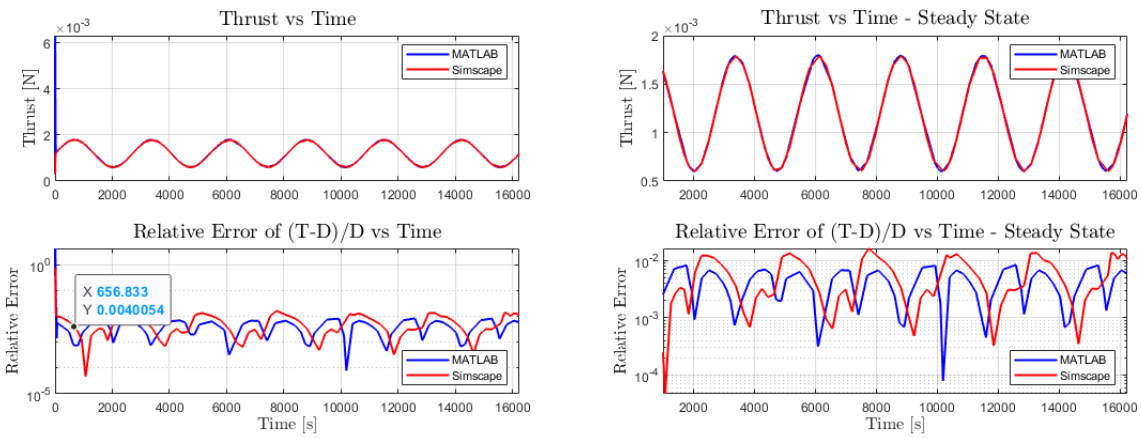
Figure 19: Comparison of thrust and relative error over time. Same parameters

- **Comparison with different parameters**

  After applying the genetic algorithm in Simscape, a comparison was performed using the optimized parameters for each simulation. This analysis highlights the differences in performance and behavior resulting from the parameter adjustments specific to each environment.

Parameter	MATLAB	Simscape
b_a (Ns/m)	1500.1719	1500
k_a (N/m)	2.179×10^{-3}	1.0322×10^{-3}
R_{in} (Ω)	0.15529	0.1065
R_f (Ω)	64017.17	76011.8041

The initial conditions used in both simulations are the same as those previously mentioned in the table 6.



(a) Full simulation. **(b)** Steady-state behavior.

Figure 20: Comparison of thrust and relative error over time. Different parameters

Similar behaviour as in the previous section is observed, but throughout the simulation the relative errors in Simscape have decreased. The mean relative error is an order of magnitude smaller in the case of Simscape.

References

[1] R. Mukhiya, M. Garg, P. Gaikwad, et al. *Electrical equivalent modeling of MEMS differential capacitive accelerometer*. In: Microelectronics Journal. Vol. 99, pp. 104770, 2020. https://doi.org/10.1016/j.mejo.2020.104770